

CSCE 735 PARALLEL COMPUTING · MODULE 03 · LECTURE NOTES

Threads and Shared Memory

Several instruction streams, one address space.

MODULE OUTCOMES

Distinguish a data race from a race condition.

Use threads and OpenMP correctly.

Explain why data races are undefined behaviour, not merely risky.

Choose between thread and task parallelism.

Detect races with tooling rather than reasoning.

1 Two kinds of race

	Data race	Race condition
Definition	Two threads access the same memory location, at least one of them writes, and there is no synchronisation ordering them.	The program's result depends on the relative <i>timing</i> of events.
Status	Undefined behaviour in C++ and most languages with a memory model.	A logic error — the program is well-defined and does the wrong thing.
Detection	Mechanically detectable. A sanitiser can find it without knowing what the program is supposed to do.	Not mechanically detectable , because determining whether a timing dependence is wrong requires knowing the intended behaviour.
Always a bug?	Yes.	No. A work-stealing scheduler's outcome depends on timing by design.

The two are constantly conflated, including in otherwise careful writing. The practical consequence of the distinction is in the third row: one class of bug is found by running a tool, and the other requires thinking about what the program is for.

A DATA RACE IS UNDEFINED BEHAVIOUR, NOT A SLIGHTLY WRONG NUMBER

The common intuition is that a racy counter will simply lose some increments — that the answer will be a bit low but otherwise sensible. **That is not what the standard says**, and the difference matters.

A data race is undefined behaviour, which means the compiler is entitled to assume it does not occur and to optimise on that assumption. It may hoist a load out of a loop, keep a value in a register across what you thought was a synchronisation point, or duplicate a read that you assumed happened once.

So racy code can produce results that no possible interleaving explains: a variable holding a value that was never written to it, a loop that never terminates because the exit condition was hoisted, or both branches of an if taking effect.

'It works in practice' is not a defence. It works with this compiler, at this optimisation level, on this architecture. It stops working when any of those changes — typically during a release build, in production, months later.

2 Writing parallel code

```
#pragma omp parallel for                                // independent iterations
for (int i = 0; i < n; i++) c[i] = f(a[i], b[i]);

double sum = 0.0;
#pragma omp parallel for reduction(+:sum)              // per-thread partials
for (int i = 0; i < n; i++) sum += a[i];

#pragma omp parallel for schedule(dynamic, 64) // uneven work
for (int i = 0; i < n; i++) variable_cost_work(i);

#pragma omp parallel
#pragma omp single
    quicksort(a, 0, n);                                // spawns #pragma omp task internally
```

USE THE REDUCTION CLAUSE; DO NOT HAND-ROLL IT

The common mistake is to write a shared accumulator protected by a critical section or a lock:

```
#pragma omp parallel for for (...) { #pragma omp critical sum += a[i]; }
```

This serialises the entire loop. Every iteration acquires a lock, so the threads execute the body one at a time — and they pay the lock overhead on top. **The result is reliably slower than the serial loop**, often by a large factor, while appearing to be parallel.

The reduction clause builds a private accumulator per thread and combines them once at the end, which has no contention during the loop and one logarithmic or linear combination afterwards. **This is also the general pattern for any associative combination** (Module 06), and recognising it is worth more than the clause itself.

Schedule	Behaviour	Appropriate when
static	The iteration space is divided into equal contiguous chunks and assigned to threads before the loop begins.	Work per iteration is uniform. No scheduling overhead at all, and good locality since each thread touches a contiguous range.
dynamic	Threads take chunks from a shared queue as they become free.	Work per iteration varies , so static assignment would leave some threads idle. Costs a queue operation per chunk.
guided	Dynamic, with chunk sizes starting large and shrinking.	Variable work, with less queue overhead than dynamic — large early chunks amortise the cost, small late ones balance the tail.
auto / runtime	Left to the compiler or set by an environment variable.	When you want to tune the schedule without recompiling, which is genuinely useful during measurement.

The chunk size is the real parameter. Too small and the queue operations dominate; too large and the final chunks cause imbalance. It depends on the per-iteration cost and should be measured rather than assumed.

3 Finding concurrency bugs

CONCURRENCY BUGS DO NOT REPRODUCE

A race may manifest in one run out of ten thousand, on one machine, under one particular load, with one compiler version and one optimisation level. The window in which the interleaving goes wrong may be a few nanoseconds wide.

So the standard testing method — run it and see whether it works — fails. Passing a thousand runs establishes very little about whether a race exists; it establishes that the window was not hit a thousand times.

And it will manifest in production, under a concurrency pattern and a load that your tests did not produce — typically at the worst moment, and typically without reproducing when investigated.

So use tools that detect the *possibility* of a race rather than its occurrence. ThreadSanitizer instruments every memory access and maintains the happens-before relation between threads; it reports a race when two accesses are unordered, **whether or not they actually raced on that particular run**. That is a fundamentally stronger result than testing, and it is why it should be used routinely rather than when a bug is suspected.

- **ThreadSanitizer** — compile with `-fsanitize=thread`. Roughly a 10× slowdown and 5× memory, and worth both. **Run your entire test suite under it**, not just a concurrency test. **Project 1 requires a clean run**, which is the minimum honest standard.
- **Helgrind and DRD** (Valgrind tools) — similar detection without recompilation, considerably slower, useful when you cannot rebuild.
- **Static analysis with annotations** — Clang's thread safety analysis lets you annotate which lock guards which data and checks it at compile time. Catches a real class of errors with no runtime cost.
- **Stress testing** with varied thread counts, oversubscription, and deliberate delay injection at suspicious points — which widens the windows and makes rare interleavings likelier.
- **Deterministic replay** where the framework supports it, so that a failure found once can be re-examined.

THE DISCIPLINE THAT PREVENTS MOST OF THIS

Share nothing by default. Give each thread its own state and combine at the end. **Code with no shared mutable state has no data races to find**, which is a far stronger position than code with carefully placed locks.

Make shared data immutable. Read-only sharing requires no synchronisation whatsoever — any number of threads may read the same data concurrently with no cost and no risk.

Where mutable sharing is genuinely necessary, own it explicitly: one lock, one documented invariant, and one well-defined region of code permitted to touch it. Scattered synchronisation around data that many parts of the program modify is where the difficult bugs live.

Most concurrency bugs are design failures rather than coding errors. No amount of careful locking rescues an architecture that shares too much mutable state, and the effort is better spent on the decomposition (Module 06) than on the locking.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Stanford CS149 — shared memory programming lectures (free)	Threads, OpenMP, and the decomposition material.
OpenMP specification and examples (free)	The examples document is unusually good and covers every construct in §2 with working code.
Boehm — Threads Cannot Be Implemented as a Library (free)	Why data races are undefined behaviour and why a language needs a memory model. The argument in §1, made properly.
ThreadSanitizer documentation (free)	How it works and how to use it. Read the happens-before explanation — it clarifies what the tool can and cannot find.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a program with a data race on a counter. Run it a thousand times and report how often the answer is wrong.
- 2 Compile the same program at `-O0` and `-O3` and compare the failure rates. Explain the difference.
- 3 Run it under ThreadSanitizer and confirm the race is reported even on runs that produced the correct answer.

- 4 Implement a reduction with a critical section and with the reduction clause. Measure both and compare against the serial loop.
- 5 Construct a loop with highly variable per-iteration cost. Measure static and dynamic scheduling, and sweep the chunk size.
- 6 Implement parallel quicksort with OpenMP tasks and compare against a parallel for over fixed partitions.
- 7 Write a race condition that is not a data race — properly synchronised, and still timing-dependent and wrong. Confirm ThreadSanitizer does not report it.
- 8 Run your Project 1 code under ThreadSanitizer and fix everything it reports.
- 9 Take a shared-mutable-state design and restructure it to share nothing. Compare the amount of synchronisation required.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Distinguish a data race from a race condition on four axes.
- 2 Why is a data race undefined behaviour rather than merely a wrong answer, and what can a compiler do with that?
- 3 Why is 'it works in practice' not a defence?
- 4 Why is a critical-section reduction slower than the serial loop?
- 5 Compare four OpenMP schedules and say what the chunk size trades.
- 6 Distinguish data from task parallelism and give an example of each.
- 7 Why does testing fail to find concurrency bugs, and what do sanitisers do instead?
- 8 Give three design principles that prevent most concurrency bugs.